

K-Means_iris

September 14, 2026

```
[2]: import pandas as pd
import matplotlib.pyplot as mp
import seaborn as sb
import os
from sklearn.cluster import KMeans
url = "https://gist.githubusercontent.com/netj/8836201/raw/iris.csv"
dataset = pd.read_csv(url)
dataset
```

```
[2]:      sepal.length  sepal.width  petal.length  petal.width  variety
0           5.1           3.5           1.4           0.2     Setosa
1           4.9           3.0           1.4           0.2     Setosa
2           4.7           3.2           1.3           0.2     Setosa
3           4.6           3.1           1.5           0.2     Setosa
4           5.0           3.6           1.4           0.2     Setosa
..          ...          ...          ...          ...          ...
145          6.7           3.0           5.2           2.3  Virginica
146          6.3           2.5           5.0           1.9  Virginica
147          6.5           3.0           5.2           2.0  Virginica
148          6.2           3.4           5.4           2.3  Virginica
149          5.9           3.0           5.1           1.8  Virginica
```

[150 rows x 5 columns]

```
[3]: fields = dataset.iloc[:, [1, 2]].values
print(fields)
```

```
[[3.5 1.4]
 [3.  1.4]
 [3.2 1.3]
 [3.1 1.5]
 [3.6 1.4]
 [3.9 1.7]
 [3.4 1.4]
 [3.4 1.5]
 [2.9 1.4]
 [3.1 1.5]
 [3.7 1.5]
```

[3.4 1.6]
[3. 1.4]
[3. 1.1]
[4. 1.2]
[4.4 1.5]
[3.9 1.3]
[3.5 1.4]
[3.8 1.7]
[3.8 1.5]
[3.4 1.7]
[3.7 1.5]
[3.6 1.]
[3.3 1.7]
[3.4 1.9]
[3. 1.6]
[3.4 1.6]
[3.5 1.5]
[3.4 1.4]
[3.2 1.6]
[3.1 1.6]
[3.4 1.5]
[4.1 1.5]
[4.2 1.4]
[3.1 1.5]
[3.2 1.2]
[3.5 1.3]
[3.6 1.4]
[3. 1.3]
[3.4 1.5]
[3.5 1.3]
[2.3 1.3]
[3.2 1.3]
[3.5 1.6]
[3.8 1.9]
[3. 1.4]
[3.8 1.6]
[3.2 1.4]
[3.7 1.5]
[3.3 1.4]
[3.2 4.7]
[3.2 4.5]
[3.1 4.9]
[2.3 4.]
[2.8 4.6]
[2.8 4.5]
[3.3 4.7]
[2.4 3.3]
[2.9 4.6]

[2.7 3.9]
[2. 3.5]
[3. 4.2]
[2.2 4.]
[2.9 4.7]
[2.9 3.6]
[3.1 4.4]
[3. 4.5]
[2.7 4.1]
[2.2 4.5]
[2.5 3.9]
[3.2 4.8]
[2.8 4.]
[2.5 4.9]
[2.8 4.7]
[2.9 4.3]
[3. 4.4]
[2.8 4.8]
[3. 5.]
[2.9 4.5]
[2.6 3.5]
[2.4 3.8]
[2.4 3.7]
[2.7 3.9]
[2.7 5.1]
[3. 4.5]
[3.4 4.5]
[3.1 4.7]
[2.3 4.4]
[3. 4.1]
[2.5 4.]
[2.6 4.4]
[3. 4.6]
[2.6 4.]
[2.3 3.3]
[2.7 4.2]
[3. 4.2]
[2.9 4.2]
[2.9 4.3]
[2.5 3.]
[2.8 4.1]
[3.3 6.]
[2.7 5.1]
[3. 5.9]
[2.9 5.6]
[3. 5.8]
[3. 6.6]
[2.5 4.5]

```
[2.9 6.3]
[2.5 5.8]
[3.6 6.1]
[3.2 5.1]
[2.7 5.3]
[3. 5.5]
[2.5 5. ]
[2.8 5.1]
[3.2 5.3]
[3. 5.5]
[3.8 6.7]
[2.6 6.9]
[2.2 5. ]
[3.2 5.7]
[2.8 4.9]
[2.8 6.7]
[2.7 4.9]
[3.3 5.7]
[3.2 6. ]
[2.8 4.8]
[3. 4.9]
[2.8 5.6]
[3. 5.8]
[2.8 6.1]
[3.8 6.4]
[2.8 5.6]
[2.8 5.1]
[2.6 5.6]
[3. 6.1]
[3.4 5.6]
[3.1 5.5]
[3. 4.8]
[3.1 5.4]
[3.1 5.6]
[3.1 5.1]
[2.7 5.1]
[3.2 5.9]
[3.3 5.7]
[3. 5.2]
[2.5 5. ]
[3. 5.2]
[3.4 5.4]
[3. 5.1]]
```

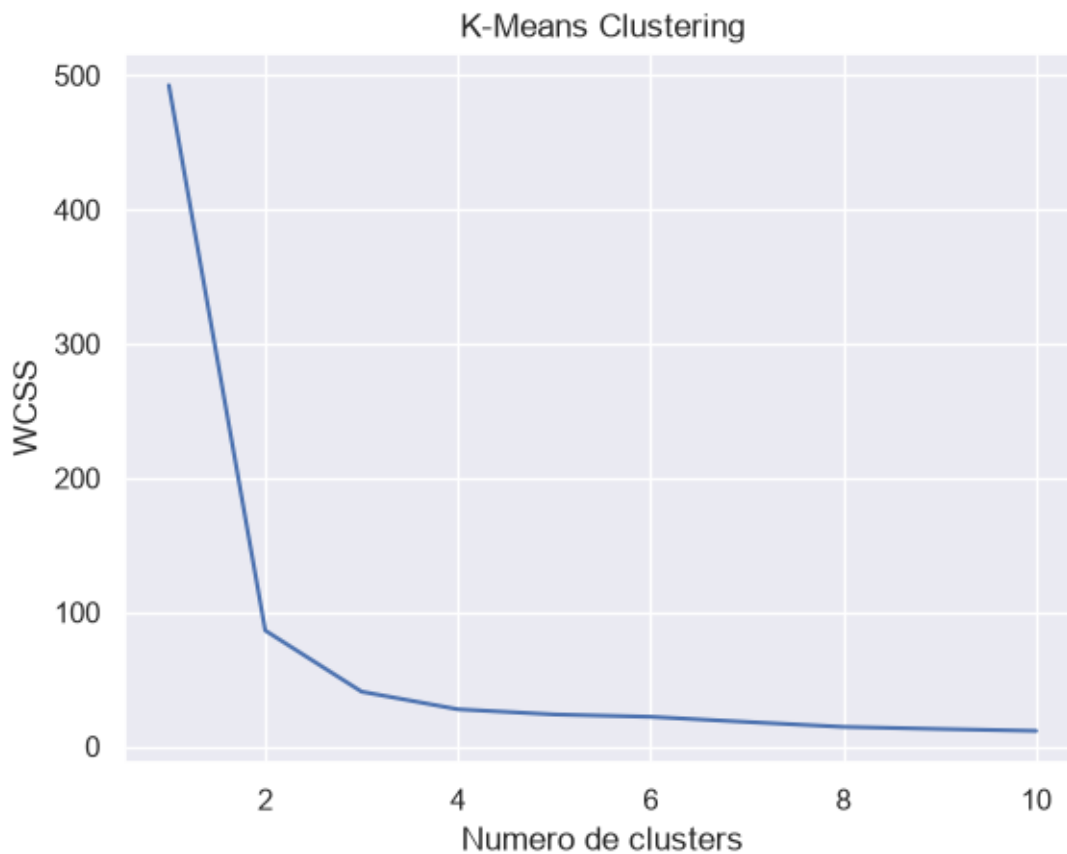
```
[4]: import warnings
      warnings.filterwarnings("ignore")
```

```
[5]: wcss = []
    for index in range(1,11):
        kmeans = KMeans(n_clusters = index, init = "k-means++", random_state = 2)
        kmeans.fit(fields)

        wcss.append(kmeans.inertia_)
```

```
[6]: ##Graficacion del "Codo de Jmabu"
    sb.set()
    mp.plot(range(1,11), wcss)
    mp.title("K-Means Clustering")
    mp.xlabel("Numero de clusters")
    mp.ylabel("WCSS") #Weigthed cluster sum of squares
```

```
[6]: Text(0, 0.5, 'WCSS')
```



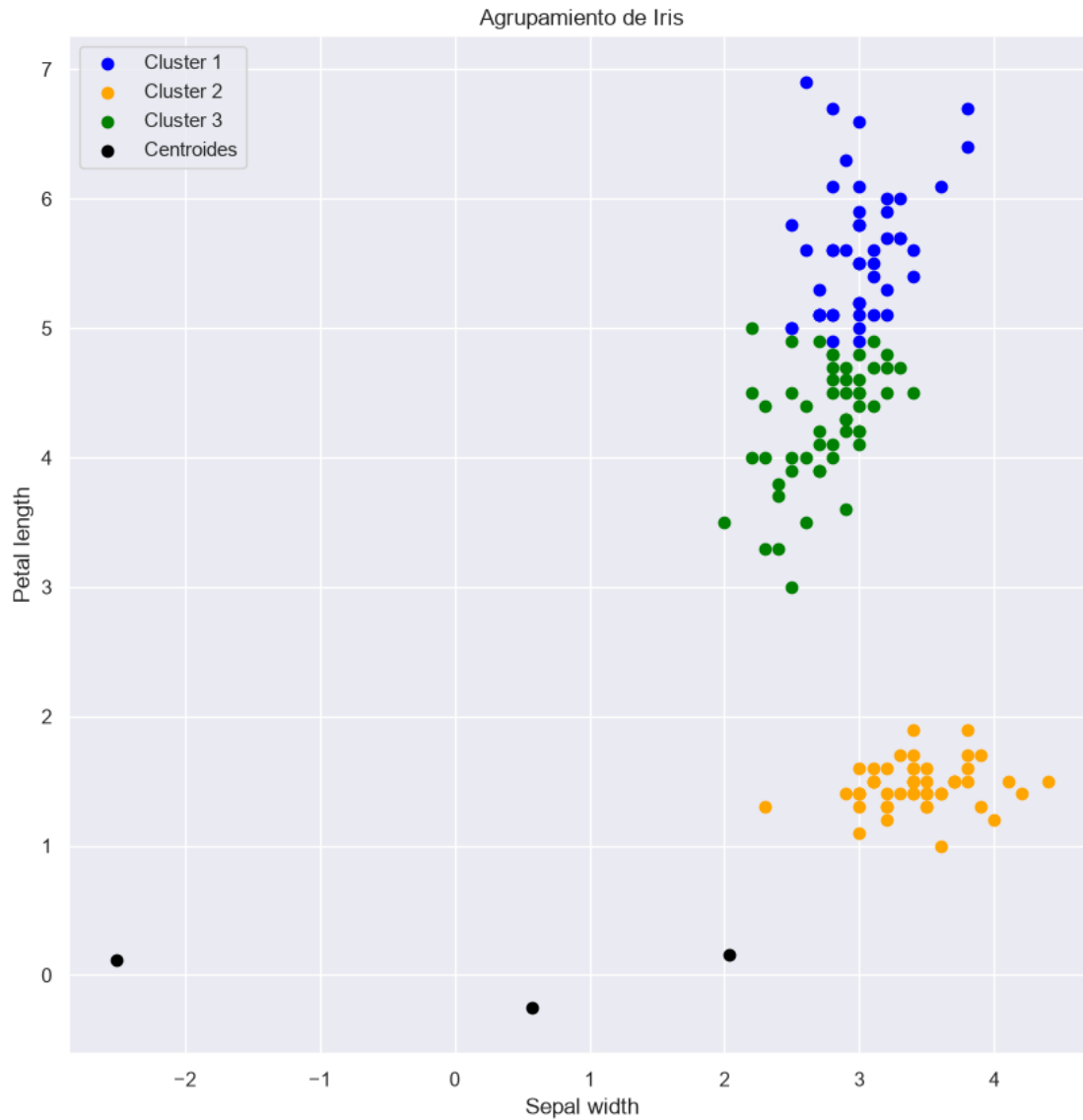
```
[7]: kmeans = KMeans(n_clusters = 3, init = "k-means++", random_state = 42)
    cluster_values = kmeans.fit_predict(fields)
    print(cluster_values)
```

[illegible]

```
[21]: mp.figure(figsize = (10,10))
      mp.scatter(fields[cluster_values == 0,0], fields [cluster_values == 0,1], c = 'blue', label = 'Cluster 1')
      mp.scatter(fields[cluster_values == 1,0], fields [cluster_values == 1,1], c = 'orange', label = 'Cluster 2')
      mp.scatter(fields[cluster_values == 2,0], fields [cluster_values == 2,1], c = 'green', label = 'Cluster 3')

      mp.scatter(kmeans.cluster_centers_[:,0],kmeans.cluster_centers_[:,1], c = 'black', label = 'Centroides')
      mp.title("Agrupamiento de Iris")
      mp.xlabel("Sepal width")
      mp.ylabel("Petal length")
      mp.legend()
```

```
[21]: <matplotlib.legend.Legend at 0x2406fe43380>
```



```
[9]: #Criterio de silueta ("Silhouette")
from sklearn import datasets
from sklearn.metrics import silhouette_score
```

```
[10]: X = dataset.iloc[:, [1,2]].to_numpy()
for j in range(2,12):
    kmeans = KMeans (n_clusters = j, random_state = 42)
    kmeans.fit_predict(X)
    #Calculamos el score de silueta
    score = silhouette_score(X, kmeans.labels_, metric = 'euclidean')
    print("Score Silhuoette;", "k =", j, ":", score)
```

Score Silhuoette; k = 2 : 0.7392862954615389

```

Score Silhuoette; k = 3 : 0.5933477102522363
Score Silhuoette; k = 4 : 0.5613370943285366
Score Silhuoette; k = 5 : 0.5068349971009903
Score Silhuoette; k = 6 : 0.37808040954390965
Score Silhuoette; k = 7 : 0.3528070329829507
Score Silhuoette; k = 8 : 0.33737507922227805
Score Silhuoette; k = 9 : 0.33316742872663524
Score Silhuoette; k = 10 : 0.34632325735288505
Score Silhuoette; k = 11 : 0.340427773629915

```

```

[11]: #Algoritmo KMeans aplicado a 3 columnas de nuestra base original
      #bajo una transformacion de PCA
      fields2 = dataset.iloc[:,[1,2,3]].values
      print(fields2)

```

```

[[3.5 1.4 0.2]
 [3.  1.4 0.2]
 [3.2 1.3 0.2]
 [3.1 1.5 0.2]
 [3.6 1.4 0.2]
 [3.9 1.7 0.4]
 [3.4 1.4 0.3]
 [3.4 1.5 0.2]
 [2.9 1.4 0.2]
 [3.1 1.5 0.1]
 [3.7 1.5 0.2]
 [3.4 1.6 0.2]
 [3.  1.4 0.1]
 [3.  1.1 0.1]
 [4.  1.2 0.2]
 [4.4 1.5 0.4]
 [3.9 1.3 0.4]
 [3.5 1.4 0.3]
 [3.8 1.7 0.3]
 [3.8 1.5 0.3]
 [3.4 1.7 0.2]
 [3.7 1.5 0.4]
 [3.6 1.  0.2]
 [3.3 1.7 0.5]
 [3.4 1.9 0.2]
 [3.  1.6 0.2]
 [3.4 1.6 0.4]
 [3.5 1.5 0.2]
 [3.4 1.4 0.2]
 [3.2 1.6 0.2]
 [3.1 1.6 0.2]
 [3.4 1.5 0.4]
 [4.1 1.5 0.1]

```

[4.2 1.4 0.2]
[3.1 1.5 0.2]
[3.2 1.2 0.2]
[3.5 1.3 0.2]
[3.6 1.4 0.1]
[3. 1.3 0.2]
[3.4 1.5 0.2]
[3.5 1.3 0.3]
[2.3 1.3 0.3]
[3.2 1.3 0.2]
[3.5 1.6 0.6]
[3.8 1.9 0.4]
[3. 1.4 0.3]
[3.8 1.6 0.2]
[3.2 1.4 0.2]
[3.7 1.5 0.2]
[3.3 1.4 0.2]
[3.2 4.7 1.4]
[3.2 4.5 1.5]
[3.1 4.9 1.5]
[2.3 4. 1.3]
[2.8 4.6 1.5]
[2.8 4.5 1.3]
[3.3 4.7 1.6]
[2.4 3.3 1.]
[2.9 4.6 1.3]
[2.7 3.9 1.4]
[2. 3.5 1.]
[3. 4.2 1.5]
[2.2 4. 1.]
[2.9 4.7 1.4]
[2.9 3.6 1.3]
[3.1 4.4 1.4]
[3. 4.5 1.5]
[2.7 4.1 1.]
[2.2 4.5 1.5]
[2.5 3.9 1.1]
[3.2 4.8 1.8]
[2.8 4. 1.3]
[2.5 4.9 1.5]
[2.8 4.7 1.2]
[2.9 4.3 1.3]
[3. 4.4 1.4]
[2.8 4.8 1.4]
[3. 5. 1.7]
[2.9 4.5 1.5]
[2.6 3.5 1.]
[2.4 3.8 1.1]

[2.4 3.7 1.]
[2.7 3.9 1.2]
[2.7 5.1 1.6]
[3. 4.5 1.5]
[3.4 4.5 1.6]
[3.1 4.7 1.5]
[2.3 4.4 1.3]
[3. 4.1 1.3]
[2.5 4. 1.3]
[2.6 4.4 1.2]
[3. 4.6 1.4]
[2.6 4. 1.2]
[2.3 3.3 1.]
[2.7 4.2 1.3]
[3. 4.2 1.2]
[2.9 4.2 1.3]
[2.9 4.3 1.3]
[2.5 3. 1.1]
[2.8 4.1 1.3]
[3.3 6. 2.5]
[2.7 5.1 1.9]
[3. 5.9 2.1]
[2.9 5.6 1.8]
[3. 5.8 2.2]
[3. 6.6 2.1]
[2.5 4.5 1.7]
[2.9 6.3 1.8]
[2.5 5.8 1.8]
[3.6 6.1 2.5]
[3.2 5.1 2.]
[2.7 5.3 1.9]
[3. 5.5 2.1]
[2.5 5. 2.]
[2.8 5.1 2.4]
[3.2 5.3 2.3]
[3. 5.5 1.8]
[3.8 6.7 2.2]
[2.6 6.9 2.3]
[2.2 5. 1.5]
[3.2 5.7 2.3]
[2.8 4.9 2.]
[2.8 6.7 2.]
[2.7 4.9 1.8]
[3.3 5.7 2.1]
[3.2 6. 1.8]
[2.8 4.8 1.8]
[3. 4.9 1.8]
[2.8 5.6 2.1]

```

[3.  5.8 1.6]
[2.8 6.1 1.9]
[3.8 6.4 2. ]
[2.8 5.6 2.2]
[2.8 5.1 1.5]
[2.6 5.6 1.4]
[3.  6.1 2.3]
[3.4 5.6 2.4]
[3.1 5.5 1.8]
[3.  4.8 1.8]
[3.1 5.4 2.1]
[3.1 5.6 2.4]
[3.1 5.1 2.3]
[2.7 5.1 1.9]
[3.2 5.9 2.3]
[3.3 5.7 2.5]
[3.  5.2 2.3]
[2.5 5.  1.9]
[3.  5.2 2. ]
[3.4 5.4 2.3]
[3.  5.1 1.8]]

```

```

[12]: from sklearn import decomposition
pca = decomposition.PCA(n_components = 2)
pca.fit(fields2)
fields2 = pca.transform(fields2)
fields2

```

```

[12]: array([[ -2.59236698e+00,  1.82790542e-01],
             [ -2.54255916e+00, -3.10617064e-01],
             [ -2.65421811e+00, -1.17396707e-01],
             [ -2.46078490e+00, -2.07792857e-01],
             [ -2.60232854e+00,  2.81472063e-01],
             [ -2.27992642e+00,  6.21236571e-01],
             [ -2.54386575e+00,  9.97549642e-02],
             [ -2.49066959e+00,  8.82517068e-02],
             [ -2.53259760e+00, -4.09298585e-01],
             [ -2.49932457e+00, -2.23438800e-01],
             [ -2.52055428e+00,  3.84296270e-01],
             [ -2.39893377e+00,  9.23943926e-02],
             [ -2.58109883e+00, -3.26263007e-01],
             [ -2.85630630e+00, -3.38691064e-01],
             [ -2.82564644e+00,  6.67912776e-01],
             [ -2.51320588e+00,  1.10635880e+00],
             [ -2.64686971e+00,  6.04665827e-01],
             [ -2.55382731e+00,  1.98436485e-01],
             [ -2.30850453e+00,  5.06909106e-01],

```

[-2.49197617e+00, 4.98623735e-01],
 [-2.30719795e+00, 9.65370785e-02],
 [-2.44347494e+00, 4.15588157e-01],
 [-2.96927183e+00, 2.64901320e-01],
 [-2.18161738e+00, 4.47933871e-02],
 [-2.12372630e+00, 1.04822450e-01],
 [-2.35908752e+00, -3.02331692e-01],
 [-2.32185443e+00, 1.23686279e-01],
 [-2.50063115e+00, 1.86933228e-01],
 [-2.58240541e+00, 8.41090210e-02],
 [-2.37901064e+00, -1.04968650e-01],
 [-2.36904908e+00, -2.03650171e-01],
 [-2.41359026e+00, 1.19543593e-01],
 [-2.59894020e+00, 7.63376412e-01],
 [-2.66209792e+00, 8.73561190e-01],
 [-2.46078490e+00, -2.07792857e-01],
 [-2.74595394e+00, -1.21539393e-01],
 [-2.68410280e+00, 1.78647856e-01],
 [-2.64086821e+00, 2.65826120e-01],
 [-2.63429499e+00, -3.14759749e-01],
 [-2.49066959e+00, 8.82517068e-02],
 [-2.64556313e+00, 1.94293800e-01],
 [-2.52602438e+00, -9.89884454e-01],
 [-2.65421811e+00, -1.17396707e-01],
 [-2.25473666e+00, 2.53659687e-01],
 [-2.08649321e+00, 5.30840421e-01],
 [-2.50401950e+00, -2.94971120e-01],
 [-2.43878002e+00, 4.87120477e-01],
 [-2.56248229e+00, -1.13254021e-01],
 [-2.52055428e+00, 3.84296270e-01],
 [-2.57244385e+00, -1.45725002e-02],
 [9.27275892e-01, 2.11205930e-01],
 [7.82343914e-01, 2.18566502e-01],
 [1.15924877e+00, 1.36455724e-01],
 [3.36239526e-01, -7.21572504e-01],
 [9.13925988e-01, -1.72016897e-01],
 [7.45110828e-01, -2.07451469e-01],
 [9.94393666e-01, 3.41179338e-01],
 [-4.31491803e-01, -6.98827614e-01],
 [8.26885089e-01, -1.04627262e-01],
 [2.43197120e-01, -3.15343162e-01],
 [-2.08173906e-01, -1.08526833e+00],
 [5.27059570e-01, 8.77540197e-03],
 [2.30582084e-01, -8.67191855e-01],
 [9.57160580e-01, -8.48386333e-02],
 [-9.04731426e-02, -1.46054121e-01],
 [6.62029985e-01, 1.00096352e-01],

[8.02267039e-01, 2.12034595e-02],
 [2.72510094e-01, -3.69641564e-01],
 [8.81959541e-01, -7.68248710e-01],
 [1.47501241e-01, -5.59644034e-01],
 [1.17317039e+00, 2.77932389e-01],
 [2.86431713e-01, -2.28164898e-01],
 [1.21901814e+00, -4.55633403e-01],
 [8.90042806e-01, -2.14812041e-01],
 [5.51677619e-01, -1.17055320e-01],
 [6.71991548e-01, 1.41483037e-03],
 [1.05885797e+00, -1.79377469e-01],
 [1.33802549e+00, 7.32087751e-02],
 [8.12228602e-01, -7.74780617e-02],
 [-2.67943282e-01, -4.93179200e-01],
 [6.57269805e-02, -6.62468241e-01],
 [-6.45485108e-02, -6.82256870e-01],
 [1.66117784e-01, -3.46635049e-01],
 [1.42110633e+00, -2.34339046e-01],
 [8.02267039e-01, 2.12034595e-02],
 [8.00960457e-01, 4.31575487e-01],
 [9.75777123e-01, 1.28170352e-01],
 [7.03182818e-01, -7.05001761e-01],
 [3.58244410e-01, -2.66591704e-02],
 [3.16316401e-01, -5.24209462e-01],
 [6.34758462e-01, -4.24603141e-01],
 [8.55463194e-01, 9.70020203e-03],
 [2.67815170e-01, -4.41173884e-01],
 [-4.21530241e-01, -7.97509135e-01],
 [4.79864921e-01, -3.18561048e-01],
 [4.11440565e-01, -3.81624278e-02],
 [4.59941796e-01, -1.21198006e-01],
 [5.51677619e-01, -1.17055320e-01],
 [-6.78121167e-01, -5.96928207e-01],
 [3.78167536e-01, -2.24022213e-01],
 [2.53381638e+00, 5.35847743e-01],
 [1.53672534e+00, -1.87401216e-01],
 [2.31780657e+00, 1.73076721e-01],
 [1.93694166e+00, 1.50293122e-02],
 [2.26461042e+00, 1.84579978e-01],
 [2.95995733e+00, 2.02075521e-01],
 [9.29154189e-01, -4.40912260e-01],
 [2.57909242e+00, 4.40281130e-02],
 [2.16025956e+00, -3.71411401e-01],
 [2.59566752e+00, 8.36034992e-01],
 [1.52545719e+00, 3.21652333e-01],
 [1.72019699e+00, -1.79115844e-01],
 [1.95086328e+00, 1.56505977e-01],

```
[ 1.50345231e+00, -3.73261001e-01],
[ 1.71946212e+00, -1.04899785e-02],
[ 1.82454784e+00,  3.76875534e-01],
[ 1.83524428e+00,  1.09568148e-01],
[ 3.01054032e+00,  1.01131632e+00],
[ 3.35209039e+00, -1.48930619e-01],
[ 1.34063866e+00, -7.47535280e-01],
[ 2.19149114e+00,  3.93446278e-01],
[ 1.38183180e+00, -8.13591232e-02],
[ 3.03307661e+00, -6.79077828e-03],
[ 1.31471402e+00, -2.11332531e-01],
[ 2.10445024e+00,  4.60835912e-01],
[ 2.27400027e+00,  3.27644619e-01],
[ 1.21301664e+00, -1.16793696e-01],
[ 1.28482934e+00,  8.47120325e-02],
[ 2.06252223e+00, -3.67143792e-02],
[ 2.03337241e+00,  9.07043185e-02],
[ 2.44412201e+00, -4.72928365e-02],
[ 2.65825352e+00,  9.67596376e-01],
[ 2.10106190e+00, -2.10684359e-02],
[ 1.37260510e+00, -1.51303468e-01],
[ 1.81266768e+00, -3.43599024e-01],
[ 2.57835755e+00,  2.12653979e-01],
[ 2.11837186e+00,  6.02312577e-01],
[ 1.82528271e+00,  2.08249669e-01],
[ 1.19309351e+00,  8.05693467e-02],
[ 1.84916589e+00,  2.51044813e-01],
[ 2.14825654e+00,  3.06268014e-01],
[ 1.65103776e+00,  2.69908642e-01],
[ 1.53672534e+00, -1.87401216e-01],
[ 2.37496278e+00,  4.01731649e-01],
[ 2.25860891e+00,  5.23419685e-01],
[ 1.75273515e+00,  1.75369806e-01],
[ 1.46491264e+00, -3.88906944e-01],
[ 1.63711614e+00,  1.28431977e-01],
[ 1.89636054e+00,  5.78381263e-01],
[ 1.46830098e+00,  9.29974042e-02]])
```

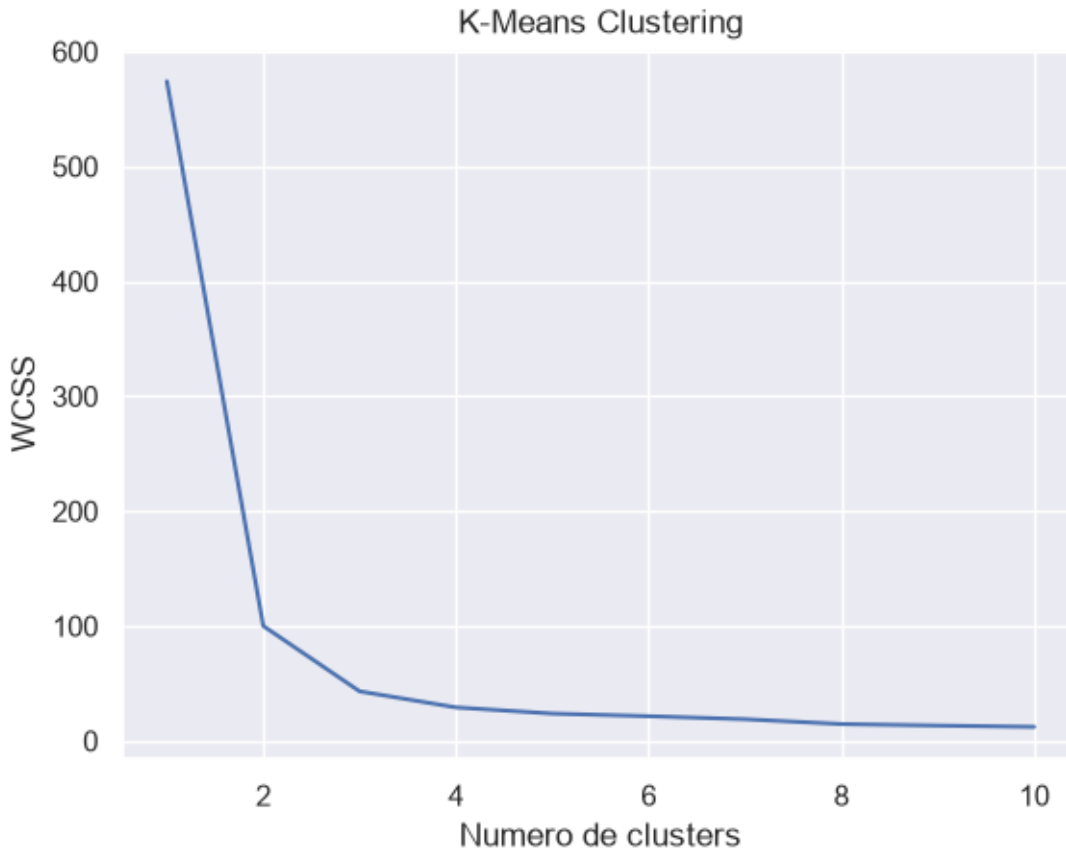
```
[17]: wcss2 = []
      for index in range(1,11):
          kmeans = KMeans(n_clusters = index, init = "k-means++", random_state = 2)
          kmeans.fit(fields2)

          wcss2.append(kmeans.inertia_)
```

```
[18]: ##Graficacion del "Codo de Jmabu"
      sb.set()
```

```
mp.plot(range(1,11), wcss2)
mp.title("K-Means Clustering")
mp.xlabel("Numero de clusters")
mp.ylabel("WCSS") #Weigthed cluster sum of squares
```

```
[18]: Text(0, 0.5, 'WCSS')
```



```
[19]: kmeans = KMeans(n_clusters = 3, init = "k-means++", random_state = 42)
      cluster_values = kmeans.fit_predict(fields2)
      print(cluster_values)
```

[illegible]

```
[23]: mp.figure(figsize = (10,10))
      mp.scatter(fields2[cluster_values == 0,0], fields2 [cluster_values == 0,1], c = 'blue', label = 'Cluster 1')
```

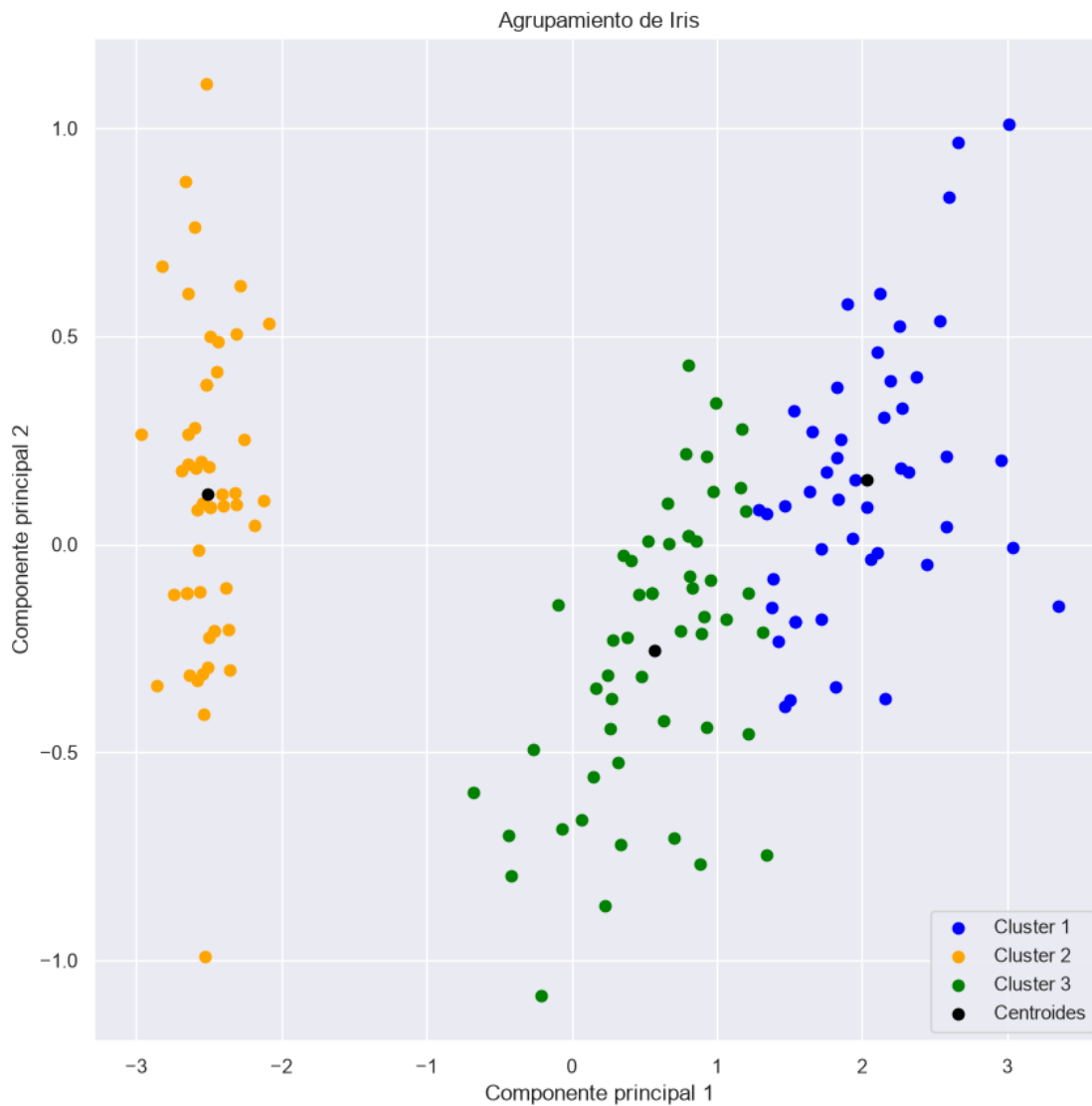
```

mp.scatter(fields2[cluster_values == 1,0], fields2 [cluster_values == 1,1], c =_
↳'orange', label = 'Cluster 2')
mp.scatter(fields2[cluster_values == 2,0], fields2 [cluster_values == 2,1], c =_
↳'green', label = 'Cluster 3')

mp.scatter(kmeans.cluster_centers_[ :,0],kmeans.cluster_centers_[ :,1], c =_
↳'black', label = 'Centroides')
mp.title("Agrupamiento de Iris")
mp.xlabel("Componente principal 1")
mp.ylabel ("Componente principal 2")
mp.legend()

```

[23]: <matplotlib.legend.Legend at 0x2406f7c12b0>



```
[24]: for j in range(2,12):
        kmeans = KMeans(n_clusters = j, random_state = 42)
        kmeans.fit_predict(fields2)
        score = silhouette_score(fields2, kmeans.labels_, metric = 'euclidean')
        print("Score Silhouette (PCA);", "k =", j, ":", score)
```

```
Score Silhouette (PCA); k = 2 : 0.7391565098868098
Score Silhouette (PCA); k = 3 : 0.6147991395660193
Score Silhouette (PCA); k = 4 : 0.564102553018288
Score Silhouette (PCA); k = 5 : 0.5282297333022685
Score Silhouette (PCA); k = 6 : 0.40595779038025454
Score Silhouette (PCA); k = 7 : 0.38851636771919557
Score Silhouette (PCA); k = 8 : 0.34534001687655175
Score Silhouette (PCA); k = 9 : 0.3631835955613131
Score Silhouette (PCA); k = 10 : 0.35779420609326423
Score Silhouette (PCA); k = 11 : 0.3439768244470373
```

```
[ ]: #¿Obtuvo los mismos resultados de agrupamiento?
#el metodo del codo nos sugiere un numero de clusters de 3 a 4, tanto con PCA
↳ como sin PCA, la forma que
#hay en la curva es casi la misma en ambos casos, lo que nos indica que
↳ aplicando PCA no se cambio el numero
#de clusters
#Para el criterio de Silhouette nos indica que k = 2, tanto con PCA como sin
↳ PCA, (score = 0.7392 y score = 0.7393)
#los valores son identicos, asi que obtuvimos los mismos resultados.
```